```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files


def run_feature_experiment(image_path):
  # 1. Image Acquisition
  # Load the image in grayscale for feature processing
  image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

  if image is None:
    print(f"Error: Could not load image at {image_path}")
    return

  # 2. Experimental Preprocessing (Simulate noise for testing)
  noisy_image = image.copy()
  num_salt = np.ceil(0.01 * image.size)
  coords = [np.random.randint(0, i - 1, int(num_salt)) for i in image.shape]
  noisy_image[coords[0], coords[1]] = 255  # Add Salt Noise

  num_pepper = np.ceil(0.01 * image.size)
  coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in image.shape]
  noisy_image[coords[0], coords[1]] = 0  # Add Pepper Noise

  # A) Noise Removal using Median Blur
  median_blur = cv2.medianBlur(noisy_image, 5)

  # B) Contrast Normalization using CLAHE
  clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
  normalized_image = clahe.apply(median_blur)

  # 3. Feature Detection (ORB Algorithm)
  orb = cv2.ORB_create()

  # Detect keypoints and descriptors
  kp_orig, des_orig = orb.detectAndCompute(image, None)
  kp_noisy, des_noisy = orb.detectAndCompute(noisy_image, None)
  kp_norm, des_norm = orb.detectAndCompute(normalized_image, None)

  # Visualize feature points by drawing keypoint circles
  img_orig_kp = cv2.drawKeypoints(
      image, kp_orig, None, color=(0, 255, 0), flags=0
  )
  img_noisy_kp = cv2.drawKeypoints(
      noisy_image, kp_noisy, None, color=(0, 255, 0), flags=0
  )
  img_norm_kp = cv2.drawKeypoints(
      normalized_image, kp_norm, None, color=(0, 255, 0), flags=0
  )

  # 4. Feature Matching Example (Original vs Normalized)
  bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
  matches = bf.match(des_orig, des_norm)
  img_matches = cv2.drawMatches(
      image, kp_orig, normalized_image, kp_norm, matches[:25], None, flags=2
  )

  # 5. Analysis & Visualization Output
  plt.figure(figsize=(7.5, 9.5))

  plt.subplot(2, 2, 1)
  plt.imshow(cv2.cvtColor(img_orig_kp, cv2.COLOR_BGR2RGB))
  plt.title(f"1. Original Image (Keypoints: {len(kp_orig)})")
  plt.axis("off")

  plt.subplot(2, 2, 2)
  plt.imshow(cv2.cvtColor(img_noisy_kp, cv2.COLOR_BGR2RGB))
  plt.title(f"2. Noisy Image (Keypoints: {len(kp_noisy)})")
  plt.axis("off")

  plt.subplot(2, 2, 3)
  plt.imshow(cv2.cvtColor(img_norm_kp, cv2.COLOR_BGR2RGB))
  plt.title(f"3. Denoised & Normalized (Keypoints: {len(kp_norm)})")
  plt.axis("off")

  plt.subplot(2, 2, 4)
  plt.imshow(cv2.cvtColor(img_matches, cv2.COLOR_BGR2RGB))
  plt.title("4. Feature Matching Across Processed Pair")
  plt.axis("off")
```

```
  plt.tight_layout()
  plt.show()


# --- Google Colab Image Upload Prompt ---
print("Click the button below to upload your input image:")
uploaded = files.upload()

# Process the first uploaded file automatically
for filename in uploaded.keys():
  print(f"Uploaded file '{filename}' with length {len(uploaded[filename])} bytes.")
  run_feature_experiment(filename)  # Run the full experiment pipeline
```

```
Click the button below to upload your input image:
```
Choose Files building image.jpg

**building image.jpg**(image/jpeg) - 40981 bytes, last modified: 10/8/2026 - 100% done

```
Saving building image.jpg to building image (1).jpg
Uploaded file 'building image (1).jpg' with length 40981 bytes.
```